

LeetCode 4057

Number of Intersecting Interval Pairs II

Medium · Weekly Contest 520 (Q2) · Sweep line + sorting · C++

1. Problem

You get n closed intervals $[\text{start}, \text{end}]$. Count the pairs (i, j) with $i < j$ whose intervals share **at least one point**. Touching at one endpoint counts: $[1,2]$ and $[2,3]$ share the point 2.

- Constraints: $2 \leq n \leq 10^5$, $0 \leq \text{start} \leq \text{end} \leq 10^9$.
- The answer can be as large as $n(n-1)/2 \approx 5 \cdot 10^9$, so it needs a 64-bit integer.

Input	Output	Why
$[[1,2], [2,3], [3,4]]$	2	$[1,2] \& [2,3]$ meet at 2, $[2,3] \& [3,4]$ meet at 3, $[1,2] \& [3,4]$ never meet
$[[1,5], [2,4], [3,6]]$	3	all three pairs overlap
$[[1,2], [3,4], [5,6]]$	0	all intervals are disjoint

2. Intuition

Obvious approach. Check every pair. Two closed intervals intersect when $\max(\text{start}_i, \text{start}_j) \leq \min(\text{end}_i, \text{end}_j)$. That is $O(n^2)$.

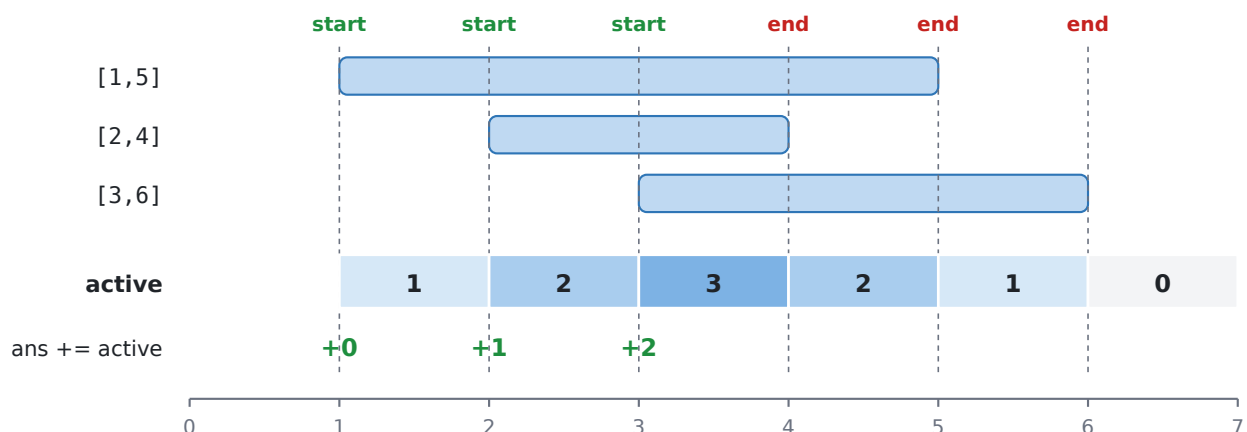
Why it is not enough. With $n = 10^5$ there are about $5 \cdot 10^9$ pairs, far too many checks. We need to count pairs without looking at each one.

The observation. Take any intersecting pair and look at the interval that **starts later**. At the moment it starts, the other interval has already started and has not ended yet (otherwise they would not touch). So:

Intersecting pair = “one interval starts while the other one is still open”.

So at every **start**, the number of intervals that are open right now is exactly the number of new pairs this interval forms with earlier ones. Every pair is counted once: at the start of the later interval.

So we sweep. Sort all start and end points, walk left to right, and keep **active** = how many intervals are open. At a start, add **active** to the answer, then that interval opens too.



Example $[[1,5], [2,4], [3,6]]$: the third interval starts at 3 while two are open (+2), the second starts at 2 while one is open (+1), the first starts on an empty line (+0). Total 3.

Why it works (short proof).

- Intersecting → counted once.** Let x be the later start. The other interval began at or before x and ends at or after x , so it is still open when x 's start is processed. It is counted there, and nowhere else.

- **Disjoint → never counted.** If A ends before B starts, A's end event comes earlier in the sorted order, so A is already closed when B starts (and B had not started when A started).
- **Same start point.** Whichever of the two is processed second sees the first one as open, so the pair is counted exactly once.

3. Approach (what the code does)

1. For every interval push two events: `{start, 0}` and `{end, 1}`. The second number is the type: 0 = start, 1 = end.
2. `sort(events)` sorts pairs by position first, then by type. At the same position all **starts (0) come before ends (1)**.
3. Sweep once: on a start do `ans += active; active++`; on an end do `active--`.
4. Return `ans` as `long long`.

Why start = 0 and end = 1? The intervals are **closed**. [1,2] and [2,3] touch at 2, so at position 2 the start of [2,3] must be handled **before** the end of [1,2]. If ends came first, the pair would be missed.

4. State / Formula (no DP here)

This is not a DP problem. It is a one-pass sweep, so there is no recursion and no table.

Name	Meaning
<code>active</code>	number of intervals that have started but not ended at the current point of the sweep
<code>ans</code>	number of intersecting pairs found so far (needs long long)
<code>events</code>	list of (position, type), type 0 = start, 1 = end

- **Transition:** start → `ans += active`, then `active += 1`. end → `active -= 1`.
- **Base case:** `active = 0`, `ans = 0`. **Order:** events sorted by (position, type), processed once left to right.
- **Formula:** answer = $\sum$ over all intervals of (number of intervals open at that interval's start).

5. Code

Exactly as in your ZIP (LC_4057/code.cpp). The line numbers on the left are only for reference.

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  class Solution {
4  public:
5      long long countIntersectingIntervals(vector<vector<int>>& intervals) {
6          vector<pair<int,int>>events;
7          for(auto &it : intervals){
8              events.push_back({it[0],0});
9              events.push_back({it[1],1});
10         }
11         sort(events.begin(),events.end());
12         long long ans = 0;
13         int active = 0;
14         for(auto & [pos,type] : events){
15             if(type==0){
16                 ans+=active;
17                 active++;
18             }else active--;
19         }
20         return ans;
21     }
22 };

```

Lines 6–10 build the $2n$ events, line 11 sorts them, lines 12–13 are the state, lines 14–19 are the sweep (line 16 is the whole counting idea, line 18 closes an interval).

6. Dry Run

Example A — intervals $[[1,5],[2,4],[3,6]]$, expected 3. Sorted events: (1,S) (2,S) (3,S) (4,E) (5,E) (6,E) (S = start, E = end).

#	event	active before	action	ans	active after
1	(1, start)	0	ans += 0	0	1
2	(2, start)	1	ans += 1	1	2
3	(3, start)	2	ans += 2	3	3
4	(4, end)	3	active--	3	2
5	(5, end)	2	active--	3	1
6	(6, end)	1	active--	3	0

Result 3 ✓

Example B (the tie-break) — intervals $[[1,2],[2,3],[3,4]]$, expected 2. Sorted events: (1,S) (2,S) (2,E) (3,S) (3,E) (4,E). Look at position 2 and 3: the start comes first.

#	event	active before	action	ans	active after
1	(1, start)	0	ans += 0	0	1
2	(2, start)	1	ans += 1 ← pair [1,2]&[2,3]	1	2
3	(2, end)	2	active--	1	1
4	(3, start)	1	ans += 1 ← pair [2,3]&[3,4]	2	2
5	(3, end)	2	active--	2	1
6	(4, end)	1	active--	2	0

Result 2 ✓. If (2,end) were processed before (2,start), active would drop to 0 and the first pair would be lost.

7. Complexity

	Cost	Reason
Time	$O(n \log n)$	sorting $2n$ events; the sweep itself is $O(n)$
Space	$O(n)$	the events vector holds $2n$ pairs
Recursion stack	none	plain loops

Overflow: ans can reach $n(n-1)/2 \approx 5 \cdot 10^9$, which is more than 2^{31} , so **ans** is **long long**. **active** is at most n and coordinates are at most 10^9 , so plain int is fine for those.

8. Quick Revision

Core idea:	Intersecting pair = the later interval starts while the other is still open. Sweep line: at every start, add the number of currently open intervals.
State:	active (open intervals), ans (long long).
Transition:	start → ans += active, active++. end → active--.
Important observation:	Sort by (position, type) with start = 0 before end = 1, so touching endpoints still count (closed intervals).
TC:	$O(n \log n)$
SC:	$O(n)$